Problem Set 1

Fundamentals of Numerical Programming I
LEONARD STORCKS
January 6, 2026

Problem 1. Short questions

(a) What is the largest number you can represent with a 2-byte **unsigned** integer (expressed in decimal)?

(b) What is the largest number you can represent with a 2-byte **signed** integer (expressed in decimal)?

(c) When running `char c = 100 * 4;` in C, -112 is stored. Why does this happen? Note a char is a single byte.

(d) What is the meaning of the machine precision ϵ?

(e) When is $a + b$ typically equal to a in limited precision floating point arithmetic?

(f) Can 0.1 be exactly represented as a (base-2) floating-point number?

Problem 2. Basler Problem

Consider the famous series

$$\sum_{k=1}^{\infty} k^{-2} = \frac{\pi^2}{6} \tag{1}$$

known as the Basler problem. We would like to approximate this series by finitely many summands.

(a) For a given finite expression $1 + \frac{1}{2^2} + \cdots + \frac{1}{n^2}$ will it make a difference if we sum the number from small to large or from large to small on the computer at finite precision?

(b) Implement both the summation from large to small and small to large in single precision (use np.float32) and compare the results to the ground truth, $\frac{\pi^2}{6}$.

Problem 3. Rewriting expressions to avoid cancellation

Consider the limit of $x \to 0$ of the function

$$f(x) = \frac{x + e^{-x} - 1}{x^2}, \quad x > 0 \tag{2}$$

(a) What is the analytical limit?

(b) Write a program which plots $f(x)$ for $10^{-11} < x < 10^{-1}$. You should observe problematic behavior. Below which upper limit do you observe this problematic behavior? For very small x, the function evaluates to zero in finite precision. Why is that?

(c) Provide a suitable fix.